UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CONTADURÍA Y ADMINISTRACIÓN

SISTEMA DE UNIVERSIDAD ABIERTA

Alumno: Emanuel Marzano Villalobos

Programación De Dispositivos Móviles

Cristian Cardoso Arellano

Grupo 9697

m1_03

30/09/2026

Enlace a repositorio de la actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M1_03

1. Introducción

Los arreglos son una de las estructuras de datos más básicas en programación: permiten guardar muchos valores del mismo tipo bajo un solo nombre y acceder a cada uno de ellos por su posición. En el desarrollo de aplicaciones es común trabajar con grandes cantidades de datos, por lo que también es importante conocer cuánto tiempo tarda un programa en procesarlos.

En esta actividad se desarrolla una aplicación Android en lenguaje Java que crea un arreglo de un millón de números enteros generados de forma aleatoria, los ordena de menor a mayor y mide el tiempo que tarda el ordenamiento. Los resultados se imprimen en la ventana Logcat de Android Studio, herramienta que se utilizó en la actividad anterior.

2. Objetivo

Ejecutar sentencias básicas de programación en lenguaje Java dentro de Android Studio mediante la creación de un arreglo de un millón de elementos enteros aleatorios, su ordenamiento y la medición del tiempo de ejecución, imprimiendo los resultados obtenidos en la ventana Logcat.

3. Desarrollo

3.1. Creación del proyecto

Se creó un nuevo proyecto desde File → New → New Project con la plantilla Empty Views Activity, el nombre M1_03 y el lenguaje Java. Al terminar la sincronización de Gradle, Android Studio generó la actividad principal MainActivity.java, donde se escribió todo el código de esta práctica dentro del método onCreate().

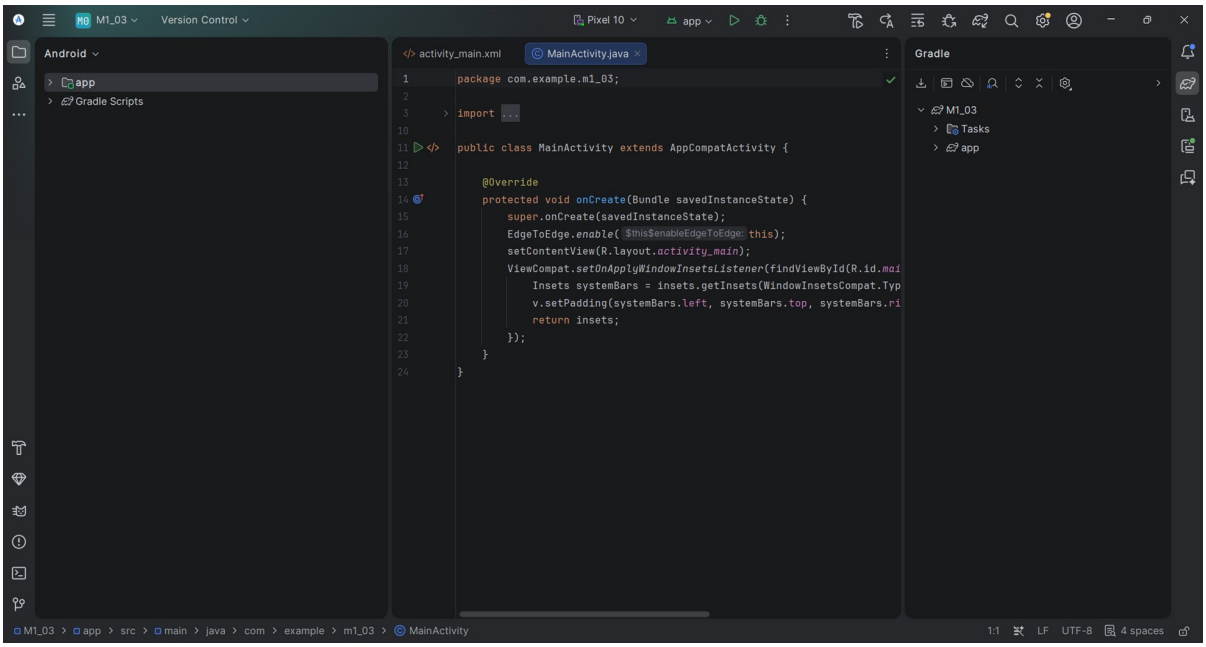


Figura 1. Proyecto M1_03 creado con la actividad MainActivity.java.

3.2. Creación y llenado del arreglo

Primero se declaró un arreglo de tipo int con capacidad para un millón de elementos y un objeto de la clase Random para generar números aleatorios. Después, con un ciclo for, se recorrió el arreglo para asignar a cada posición un número aleatorio entre 0 y 999,999.

```
int[] numeros = new int[1000000];
Random random = new Random();

for (int i = 0; i < numeros.length; i++) {
    numeros[i] = random.nextInt(1000000);
}
```

- **int[] numeros = new int[1000000]:** crea el arreglo con un millón de posiciones.
- **Random:** clase de Java (java.util.Random) que genera números aleatorios.
- **numeros.length:** devuelve el tamaño del arreglo y sirve como límite del ciclo.
- **random.nextInt(1000000):** genera un número aleatorio entre 0 y 999,999.

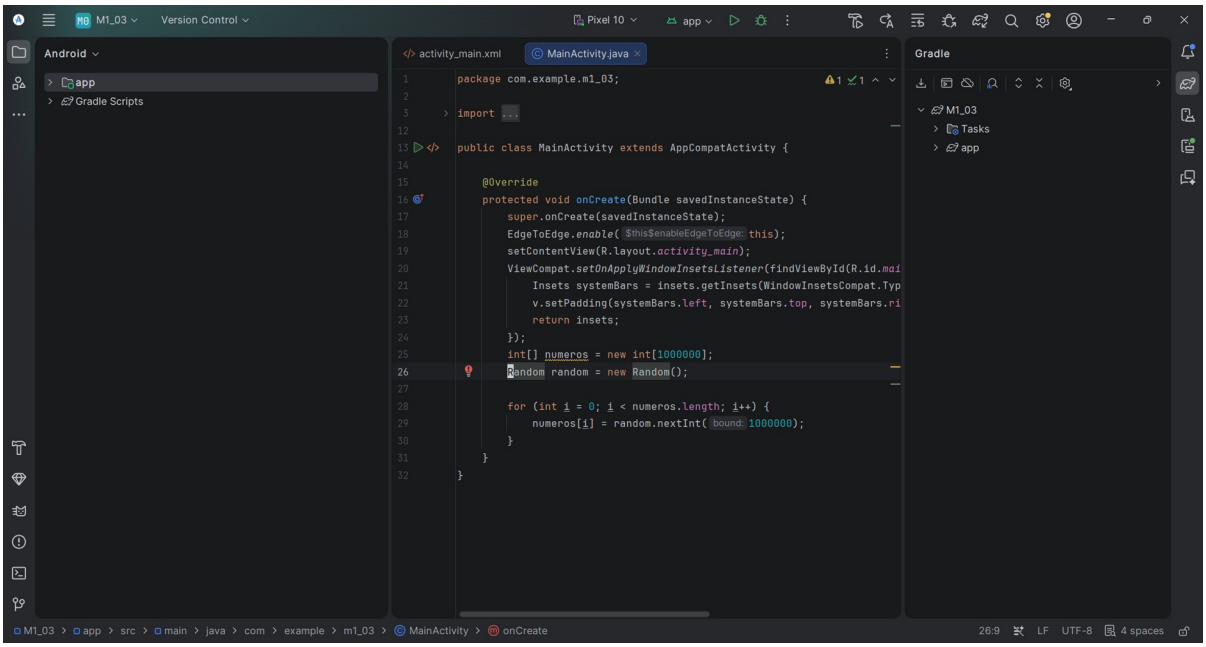


Figura 2. Declaración del arreglo y ciclo for para llenarlo con números aleatorios.

3.3. Ordenamiento y medición del tiempo

Antes de ordenar se imprimió en Logcat el total de elementos y los primeros diez números del arreglo, para poder compararlos después. Para medir el tiempo se registró la hora del sistema en milisegundos antes y después del ordenamiento, y se calculó la diferencia.

```
Log.d("M1_03", "Total de elementos: " + numeros.length);
Log.d("M1_03", "Antes de ordenar: " +
Arrays.toString(Arrays.copyOf(numeros, 10)));

long inicio = System.currentTimeMillis();
Arrays.sort(numeros);
long fin = System.currentTimeMillis();
long tiempo = fin - inicio;
```

- **Arrays.copyOf(numeros, 10):** toma solo los primeros diez elementos, ya que imprimir el millón completo saturaría Logcat.
- **Arrays.toString(...):** convierte esos elementos en texto con el formato [a, b, c, ...].
- **System.currentTimeMillis():** devuelve la hora actual en milisegundos; se guarda en variables long porque es un número muy grande.
- **Arrays.sort(numeros):** método de Java que ordena el arreglo de menor a mayor.

Durante el desarrollo se presentó un error de estructura: las nuevas líneas quedaron fuera del método onCreate() porque la llave de cierre del método estaba antes del código agregado, lo que generó 20 errores de compilación.

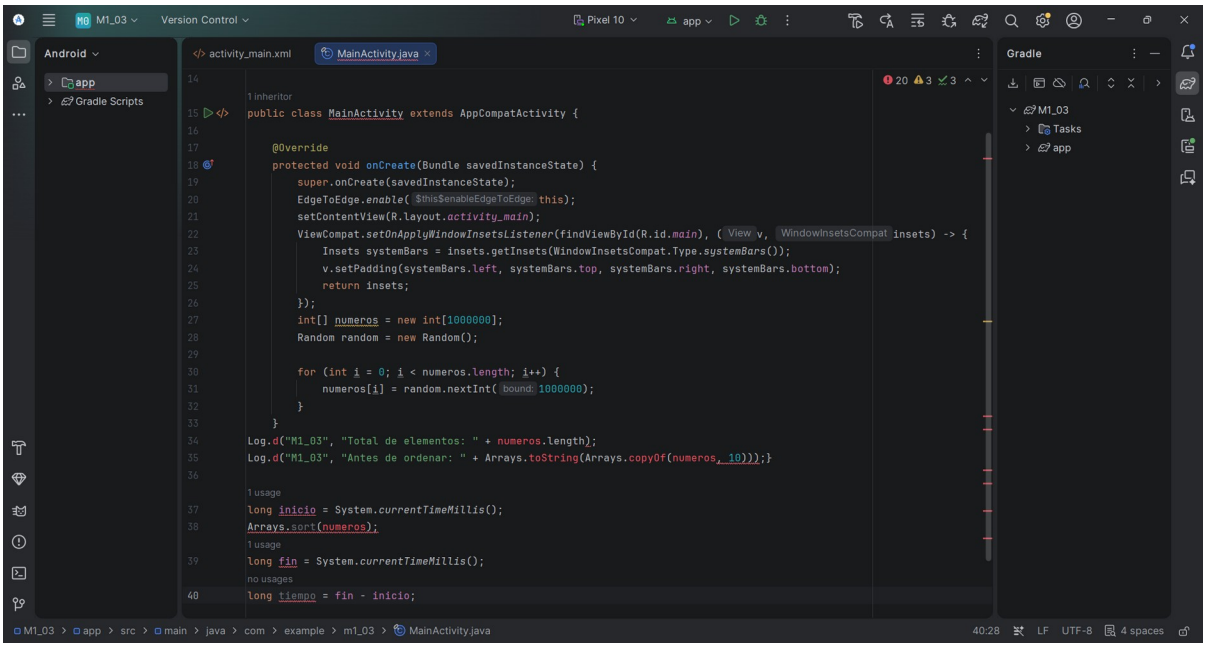


Figura 3. Errores de compilación causados por llaves mal colocadas.

Para corregirlo se eliminó la llave que cerraba el método antes de tiempo y la que había quedado al final de la línea 35, y se colocaron las llaves de cierre del método y de la clase al final del código. Con esto los errores desaparecieron.

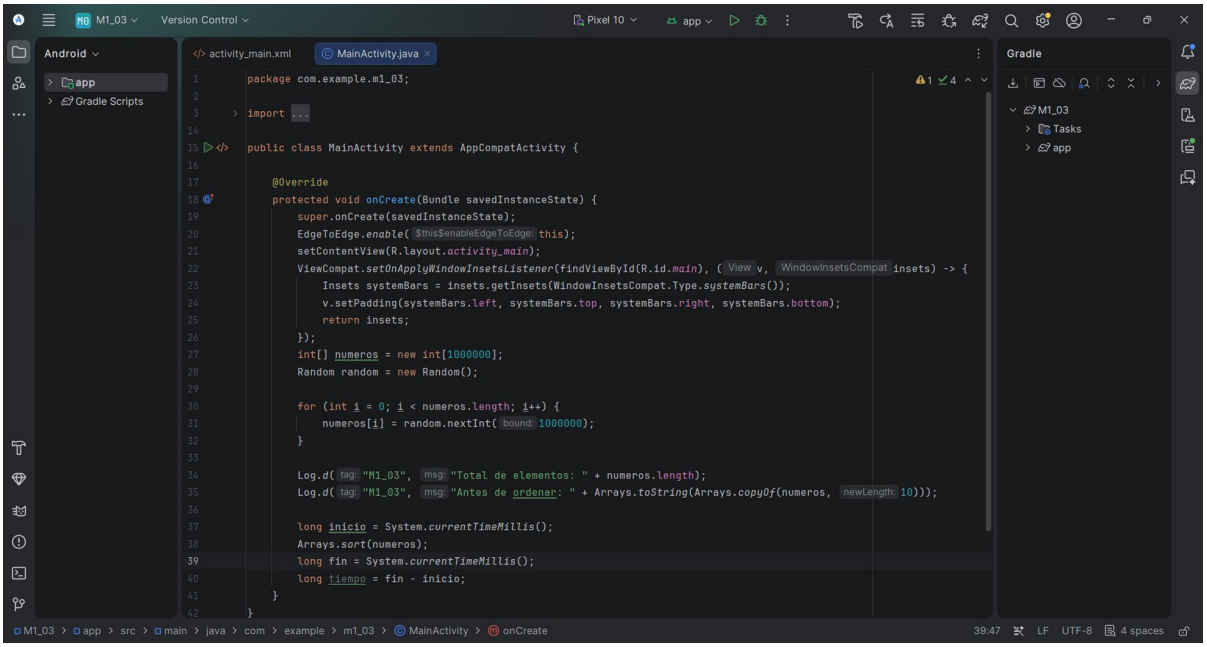


Figura 4. Código corregido, sin errores de compilación.

3.4. Impresión de resultados

Finalmente, después del ordenamiento, se imprimieron en Logcat los primeros diez elementos ya ordenados y el tiempo que tardó el proceso.

```
Log.d("M1_03", "Después de ordenar: " +
Arrays.toString(Arrays.copyOf(numeros, 10)));
Log.d("M1_03", "Tiempo de ordenamiento: " + tiempo + " ms");
```

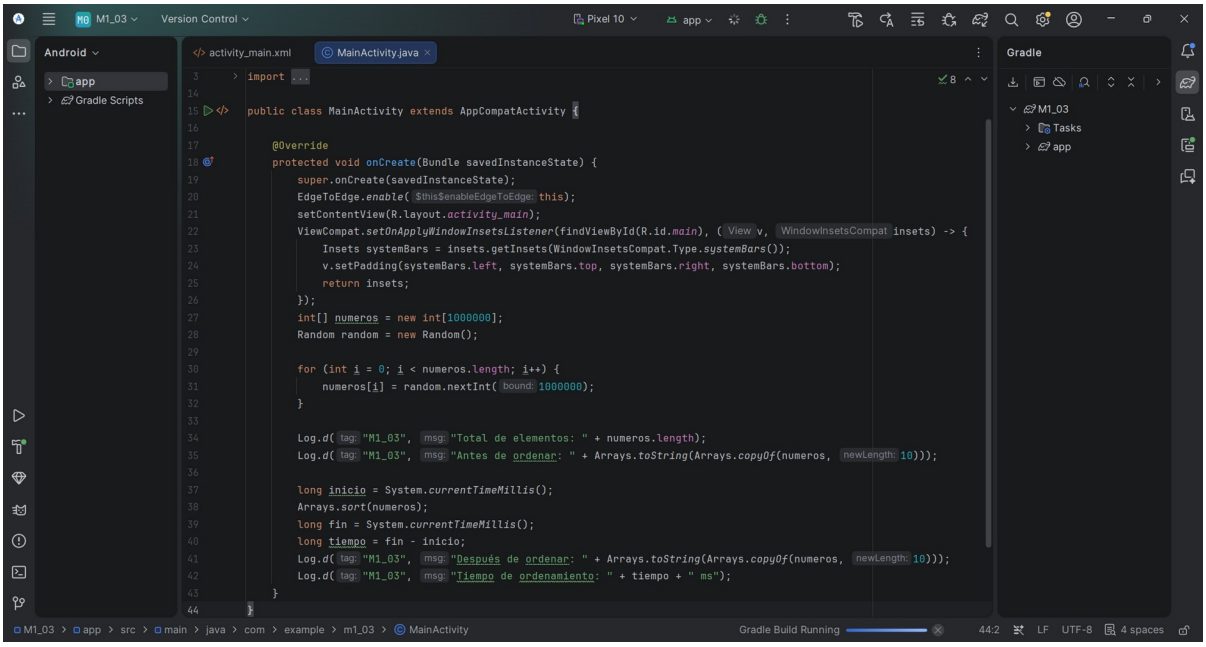


Figura 5. Código completo de MainActivity.java.

3.5. Ejecución y resultados en Logcat

La aplicación se ejecutó en el emulador Pixel 10 (Android 17, API 37). Como en esta actividad no se modificó el diseño, la pantalla muestra el texto “Hello World!” que incluye la plantilla; los resultados se consultan en Logcat.

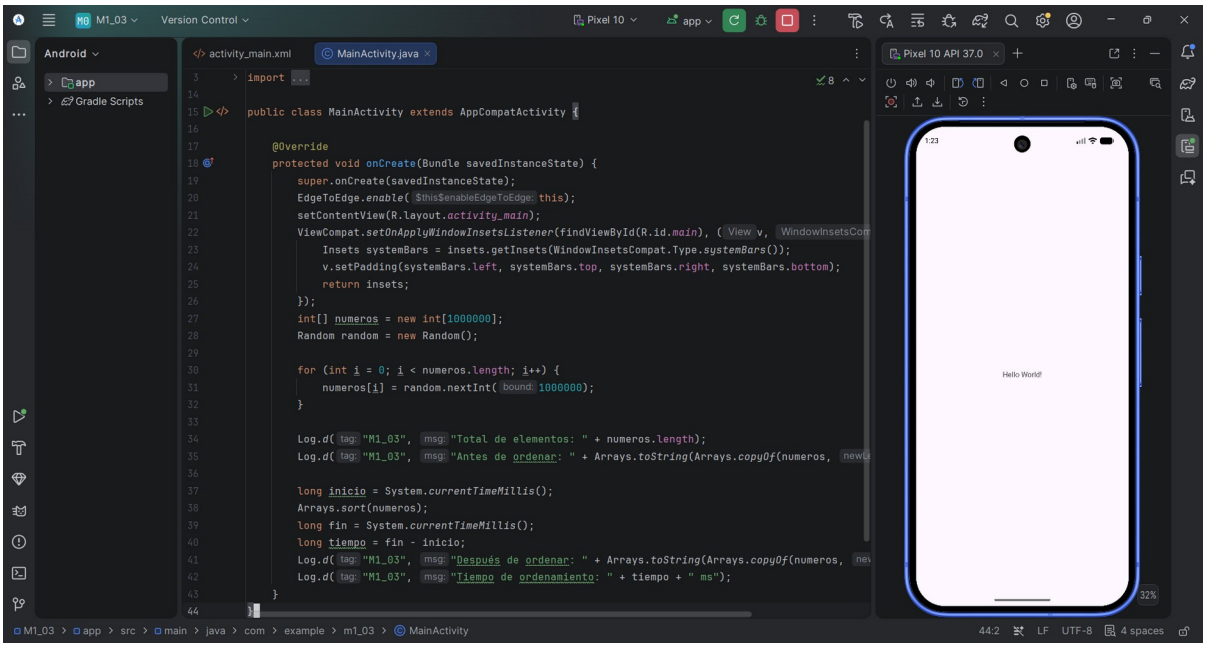


Figura 6. Aplicación en ejecución en el emulador.

En la ventana Logcat se aplicó el filtro tag:M1_03 para mostrar únicamente los mensajes de esta práctica.

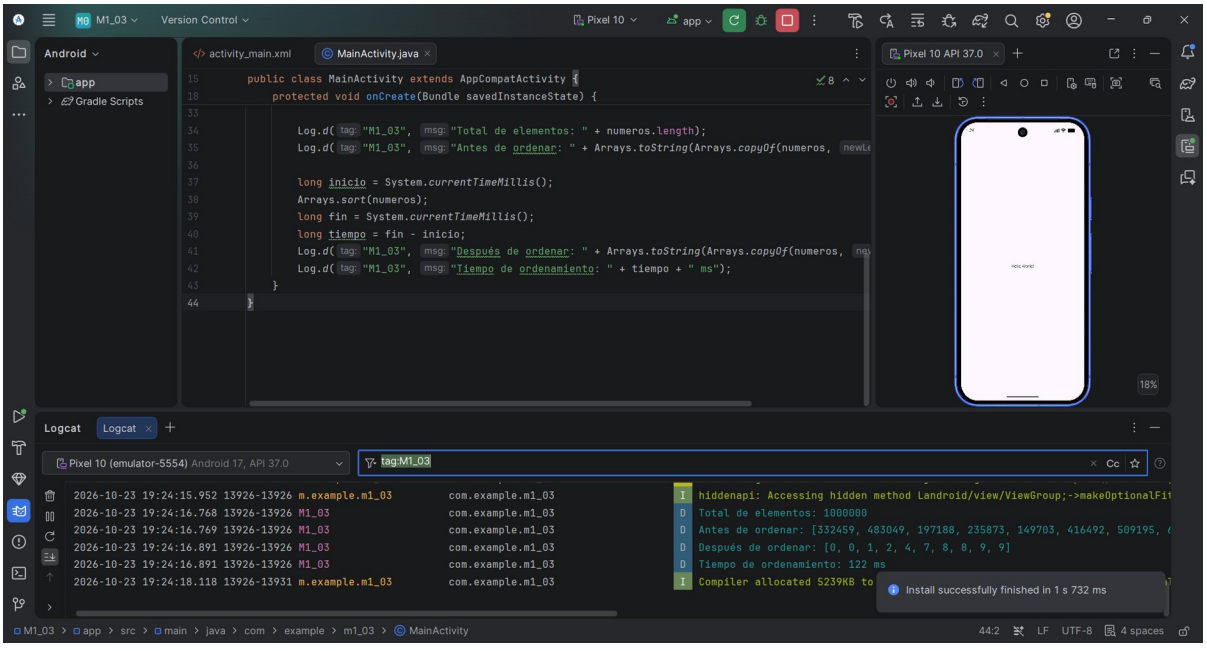


Figura 7. Resultados de la ejecución en Logcat.

Descripción: Logcat muestra que el arreglo contiene 1,000,000 de elementos; antes de ordenar, los primeros números aparecen en desorden (332459, 483049, 197188…), y después del ordenamiento aparecen de menor a mayor (0, 0, 1, 2, 4, 7, 8, 8, 9, 9). Los valores repetidos son normales, ya que al generar un millón de números aleatorios en un rango de un millón es muy probable que algunos

coincidan. El ordenamiento tardó 122 milisegundos. Las líneas marcadas con I corresponden a mensajes del sistema que coinciden con el nombre del paquete y no forman parte de la práctica.

Código fuente de esta actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M1-03

4. Conclusión

Ordenar un millón de números no me sorprendió tanto, porque ya había hecho algo parecido en lenguaje C. Lo que sí me costó fue pasarlo a Java, donde la forma de declarar el arreglo, generar números al azar y medir el tiempo es diferente. Al final, Java ya trae Arrays.sort(), que ordena todo con una sola línea, y el proceso tardó alrededor de 122 milisegundos.

La parte más frustrante fue cuando una llave quedó fuera de lugar y aparecieron veinte errores al mismo tiempo. Eso es abrumador, porque no sabía por dónde empezar a revisar. Me quedó claro que al escribir código hay que ser muy cuidadoso, y que acomodar la sangría con Ctrl + Alt + L ayuda mucho a ver dónde está el problema.

También me quedó más claro qué es un arreglo: una lista de datos guardados bajo un solo nombre, donde cada dato tiene su posición, como una fila de casilleros numerados. Así no hace falta crear una variable para cada número; con un solo arreglo guardé un millón.

5. Referencias

Aula en la nube. (2022a, 13 de diciembre). *JAVA: Copiar y clonar Arrays* [Video]. YouTube. https://www.youtube.com/watch?v=AXcdQEt_LEs

Aula en la nube. (2022b, 14 de diciembre). *JAVA: Ordenar Arrays - burbuja* [Video]. YouTube. https://www.youtube.com/watch?v=7qvFbFzmqBk

Charly Cimino. (2023, 21 de diciembre). *Ciclo FOR en Java* [Video]. YouTube. https://www.youtube.com/watch?v=59gZwkeGjTI

codigofacilito. (2011, 16 de diciembre). *Tutorial Java - 9. Arreglos* [Video]. YouTube. https://www.youtube.com/watch?v=mEvjuG_tbqY

DiscoDurodeRoer. (2024, 10 de julio). *ARRAYS UTIL | Curso Java SE #48* [Video]. YouTube. https://www.youtube.com/watch?v=zBxgqMGzeVU

GeeksforGeeks. (2025, 23 de julio). *Java.lang.System.currentTimeMillis() method with examples*. https://www.geeksforgeeks.org/java/java-lang-system-currenttimemillis-method-with-examples/

Google. (s. f.). *View logs with Logcat*. Android Developers. https://developer.android.com/studio/debug/logcat

Haniel, J. (2024, 30 de enero). *Clase Java.Util.Random*. CodeGym. https://codegym.cc/es/groups/posts/es.825.clase-java-util-random

Lideratec LP Academy. (2026, 10 de julio). *Arreglos bidimensionales en Java: índices y memoria* [Video]. YouTube. https://www.youtube.com/watch?v=LKFZh7inoQA

López, Á. (2026, 21 de febrero). *Clases en Java de utilidad*. Ciberaula. https://www.ciberaula.com/cursos/java/clases_java_utilidad.php

Oracle. (s. f.-a). *Arrays (Java SE 17 & JDK 17)*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html

Oracle. (s. f.-b). *Random (Java SE 17 & JDK 17)*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Random.html

Schults, C. (2024, 19 de marzo). *Here's how to calculate elapsed time in Java*. Stackify. https://stackify.com/heres-how-to-calculate-elapsed-time-in-java/

THE SEGA RED. (2023, 12 de marzo). *Ciclo FOR + ejercicios (Estructuras de repetición) - Curso de programación en C #15* [Video]. YouTube. https://www.youtube.com/watch?v=zygo58I7dJ8